Pharmacy Microservices - Complete Testing Documentation Index

Quick Start

30-Second Setup

```
# 1. Start all services
cd C:\RN\Microservice-API-Gateway && npm run dev
cd C:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN && npm run dev
cd C:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN-2 && npm run dev

# 2. Import Postman Collection
# File: Pharmacy_API_Collection.postman_collection.json

# 3. Start testing
# See POSTMAN_API_TESTING_GUIDE.md
```

Testing Documentation Files

1. **POSTMAN_API_TESTING_GUIDE.md** START HERE

Location: `c:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN\POSTMAN_API_TESTING_GUIDE.md`

What it contains:

- Complete API documentation (16 endpoints)
- Request/response examples for each endpoint
- Required headers and query parameters
- Authentication setup
- Error codes and troubleshooting
- Testing workflow recommendations

When to use:

- First time setting up tests
- Need endpoint details
- Debugging API behavior
- Understanding request/response format

Key sections:

```
├── Environment Setup
├── Authentication
```

```
├── Prescription APIs (2 endpoints)
├── Search APIs (2 endpoints)
├── Bucket/Cart APIs (3 endpoints)
├── Medicine Store APIs (8 endpoints)
├── Error Handling
└── Troubleshooting
```

---

2. **Pharmacy_API_Collection.postman_collection.json** ⭐ IMPORT THIS

Location: `c:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN\Pharmacy_API_Collection.postman_collection.json`

What it contains:

- Pre-configured Postman collection with all 16 APIs
- Environment variables setup
- Request templates with example data
- Organized into 5 folders for easy navigation
- Test flow examples

How to use:

1. Open Postman
2. Click **Import** button
3. Select file → `Pharmacy_API_Collection.postman_collection.json`
4. Click **Import**
5. Select environment with correct gateway_url
6. Start testing!

Included folders:

```
├── 📄 Prescription APIs (2 tests)
├── 🔍 Search APIs (2 tests)
├── 🛒 Bucket/Cart APIs (3 tests)
├── 🏪 Medicine Store APIs (8 endpoints)
└── 🧪 Test Flow Examples (5-step workflow)
```

---

3. **TESTING_CURL_REFERENCE.md**

Location: `c:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN\TESTING_CURL_REFERENCE.md`

What it contains:

- CURL command examples for all 16 APIs
- Environment variable setup
- Batch testing scripts

- JWT token generation commands
- Debugging commands with verbose output
- Performance measurement examples
- Complete workflow scripts

When to use:

- Testing from command line / bash
- CI/CD pipeline integration
- Scripted testing and automation
- Server environment testing (no GUI)
- Quick verification without Postman

Key sections:

```
├── Environment Variables setup
├── Individual API CURL commands
├── Complete Testing Workflow (5 steps)
├── Batch Testing Script (shell)
├── JWT Token Generation
├── Debugging Commands
└── Performance Analysis
```

Quick example:

```
curl -X GET "http://localhost:3000/v2/medicine-store/all" \
  -H "Authorization: Bearer $AUTH_TOKEN"
```

---

4. **TESTING_CHECKLIST.md** ⭐ USE FOR SYSTEMATIC TESTING

Location: `c:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN\TESTING_CHECKLIST.md`

What it contains:

- Pre-testing setup verification (9 items)
- 27 detailed test cases organized by API group
- Specific assertions and expected results
- Performance benchmarks
- Security and edge case testing
- Integration test scenarios
- Final verification checklist

When to use:

- Systematic testing of all APIs
- Quality assurance verification

- Test coverage validation
- Performance baseline establishment
- Pre-production signoff

Included test groups:

```
├── Pre-Testing Setup (9 checks)
├── Prescription APIs (2 tests)
├── Search APIs (2 tests)
├── Bucket/Cart APIs (3 tests)
├── Medicine Store APIs (9 tests)
├── Integration Tests (3 tests)
├── Security Tests (3 tests)
├── Performance Tests (3 tests)
├── Error Handling Tests (2 tests)
└── Final Verification
```

Test checklist structure:

```
✓ Request Validation
✓ Response Validation
✓ Data Accuracy
✓ Performance Check
✓ Error Cases
```

🔗 Architecture Overview

```
Client (Postman/CURL)
    ↓
    ├→ API Gateway (localhost:3000)
    │       ├→ /api/prescription/* → OCR Service
    │       ├→ /api/search/* → OCR Service
    │       └→ /v2/medicine-store/* → Store Service
    │
    ├→ OCR Service (localhost:5000)
    │       ├→ Database (MongoDB)
    │       ├→ Cache (Redis)
    │       └→ gRPC Client ↔ Store Service
    │
    └→ Medical Store Service (localhost:5000)
            ├→ Database (MongoDB)
            ├→ gRPC Server (port 50051)
            └→ HTTP Endpoints
```

API Endpoint Summary

#	Method	Route	Auth	Response Time
1	POST	`/api/prescription/upload`	✓	15-25s
2	POST	`/api/prescription/upload-stream`	✓	Streaming
3	POST	`/api/search`	✓	< 5s
4	GET	`/api/search/global`	✗	< 3s
5	GET	`/api/bucket`	✓	< 1s
6	POST	`/api/bucket/add`	✓	< 1s
7	POST	`/api/bucket/remove`	✓	< 1s
8	GET	`/v2/medicine-store/all`	✗	< 3s
9	GET	`/v2/medicine-store/:id`	✗	< 1s
10	GET	`/v2/medicine-store/:id/items`	✓	< 2s
11	GET	`/v2/medicine-store/:id/items/search`	✓	< 2s
12	GET	`/v2/medicine-store/:id/review-stats`	✗	< 1s
13	GET	`/v2/medicine-store/:id/items/available`	✓	< 1s
14	GET	`/v2/medicine-store/:id/hours`	✗	< 1s
15	GET	`/v2/medicine-store/:id/inventory-status`	✗	< 1s
16	POST	`/v2/medicine-store/:id/review`	✓	< 1s

Auth Legend: ✓ = Required, ✗ = Not Required

Recommended Testing Sequence

Phase 1: Basic Connectivity (10 min)

1. ☐ Verify all 3 services are running
2. ☐ Test gateway health endpoint
3. ☐ Verify database connections

Run:

```
# Check services
curl http://localhost:3000
curl http://localhost:5000
```

Phase 2: Public APIs (15 min)

1. ☐ Test 8: Get All Stores
2. ☐ Test 9: Get Store Details
3. ☐ Test 4: Global Medicine Search
4. ☐ Test 12: Get Review Stats

No authentication needed

Phase 3: Prescription Flow (30 min)

1. ☐ Test 1: Upload Prescription
2. ☐ Test 3: Search by Prescription
3. ☐ Test 5: Get Bucket
4. ☐ Test 6: Add to Bucket
5. ☐ Test 7: Remove from Bucket

Prerequisites: Valid JWT token

Phase 4: Store Operations (20 min)

1. ☐ Test 10: Get Store Items
2. ☐ Test 11: Search in Store
3. ☐ Test 13: Available Medicines
4. ☐ Test 14: Store Hours
5. ☐ Test 15: Inventory Status
6. ☐ Test 16: Give Review

Phase 5: Integration & Performance (15 min)

1. ☐ Complete end-to-end user journey
2. ☐ Load test with concurrent requests
3. ☐ Verify cache performance
4. ☐ Check response times

Phase 6: Security & Edge Cases (10 min)

1. ☐ Test invalid JWT tokens
2. ☐ Test missing required fields
3. ☐ Test file upload limits
4. ☐ Test special characters in input

🛠️ Tools Required

Essential

- ☑ Postman (https://www.postman.com/)
- ☑ Node.js & npm
- ☑ MongoDB (running)
- ☑ Git bash or Terminal

Optional but Recommended

- [ ] Redis (for caching tests)
- [ ] Elasticsearch (for search tests)
- [ ] Postman CLI (for CI/CD)
- [ ] jq (JSON parser for CURL responses)

📋 Test Data Preparation

Sample Prescription Image

- Format: JPEG or PNG
- Size: < 5MB
- Resolution: At least 200x200px
- Content: Medicine prescription with clear text

Sample User Data

```
{
  "userId": "507f1f77bcf86cd799439011",
  "email": "test@example.com",
  "role": "customer"
}
```

Sample Store Data

```
{
  "storeId": "507f1f77bcf86cd799439012",
  "storeName": "City Pharmacy",
  "latitude": 19.0765,
  "longitude": 72.8765
}
```

🔐 Authentication Setup

JWT Token for Testing

Option 1: Use pre-generated token

```
Header: "Authorization: Bearer {{auth_token}}"
```

Option 2: Generate fresh token (Node.js)

```
node -e "
const jwt = require('jsonwebtoken');
const token = jwt.sign(
  { _id: '507f1f77bcf86cd799439011', email: 'test@example.com', role:
'customer' },
  'YOUR_JWT_SECRET',
  { expiresIn: '120d' }
);
console.log(token);
"
```

Option 3: Use Postman pre-request script

Add to Pre-request tab in Postman:

```
pm.environment.set("auth_token", "your_token_here");
```

🎯 Testing Workflow

Morning Standup Check (5 min)

```
# Quick health check
curl http://localhost:3000
curl http://localhost:5000
curl http://localhost:5000/v2/medicine-store/all
# All 200 OK? Ready to test
```

Full Test Suite (2 hours)

```
# Run checklist from TESTING_CHECKLIST.md
# Complete all 27 tests
# Record results
```

Continuous Testing (During Development)

```
# After code changes
npm run build
npm run test

# Run quick smoke tests
./test_apis.sh
```

🐛 Debugging Guide

If Test Fails

1. **Check Service Logs**

```
# Gateway
cd C:\RN\Microservice-API-Gateway
npm run dev
# Look for errors in console

# OCR Service
cd C:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN
npm run dev

# Store Service
cd C:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN-2
npm run dev
```

2. **Verify Request Format**
   - Use POSTMAN_API_TESTING_GUIDE.md
   - Compare your request with example
3. **Check Environment Variables**

```
# In Postman
# Click environment dropdown
# Verify all variables are set correctly
```

4. **Test Database Connection**

```
# Connect to MongoDB
mongosh "mongodb://localhost:27017"
use pharmacy_db
show collections
```

5. **Enable Verbose Logging**
   - Add `DEBUG=*` to environment
   - Check for detailed error messages

📊 Performance Baselines

Operation	Target	Acceptable	Alert
Upload Prescription	20s	< 30s	> 40s
Search 1000 medicines	3s	< 5s	> 10s
Get All Stores (100+)	2s	< 3s	> 5s
Add to Bucket	0.5s	< 1s	> 2s
Concurrent 10 uploads	30s	< 50s	> 60s

📞 Support Resources

Documentation Files Location

```
c:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN\
├── POSTMAN_API_TESTING_GUIDE.md (Complete API docs)
├── Pharmacy_API_Collection.postman_collection.json (Import this!)
├── TESTING_CURL_REFERENCE.md (CURL examples)
├── TESTING_CHECKLIST.md (Test cases)
└── API_TESTING_INDEX.md (This file)
```

Code Repository Locations

```
c:\RN\
├── Microservice-API-Gateway\ (Port 3000)
├── PHRMA-PRODUCTION-APP-BACKEND-MAIN\ (Port 5000 - OCR Service)
└── PHRMA-PRODUCTION-APP-BACKEND-MAIN-2\ (Port 5000 - Store Service)
```

Help & FAQ

Q: How do I get started?
A: Import `Pharmacy_API_Collection.postman_collection.json` into Postman and follow POSTMAN_API_TESTING_GUIDE.md

Q: What if services won't start?
A: Check ports 3000, 5000, 50051 are available. Kill any existing processes and restart.

Q: How do I get a valid JWT token?
A: See "JWT Token for Testing" section above or ask backend team.

Q: Can I test without Postman?
A: Yes! Use CURL commands from TESTING_CURL_REFERENCE.md

Q: How long should testing take?
A: ~2 hours for full suite, ~30 min for quick smoke tests

☑ Pre-Testing Checklist

Before starting any testing:

- ☐ All 3 services are running (check with curl)
- ☐ MongoDB is running and accessible
- ☐ Redis is running (if using caching tests)
- ☐ Postman is installed and updated
- ☐ Valid JWT token is available
- ☐ Test prescription image is ready
- ☐ Environment variables are set in Postman
- ☐ Network connectivity is stable
- ☐ Disk space is available for logs
- ☐ Browser console is open to check for errors

🎓 Learning Resources

Related Files (for reference)

- `c:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN\Databases\Schema\prescription.Schema.ts`
- `c:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN\Databases\Schema\ocrHistory.Schema.ts`
- `c:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN\Services\aggregation.service.ts`
- `c:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN\Routers\Routers\prescription.Routes.ts`

External Documentation

- Postman Learning Center
- Express.js Documentation
- MongoDB Documentation
- gRPC Documentation

🎯 Next Steps

1. **Read:** POSTMAN_API_TESTING_GUIDE.md (20 min)
2. **Setup:** Import Pharmacy_API_Collection.postman_collection.json (5 min)
3. **Test:** Follow Testing Workflow above (2 hours)
4. **Track:** Use TESTING_CHECKLIST.md to mark progress
5. **Report:** Document findings and issues found

📊 Test Results Template

```
Test Date: _____
Tester: _____
Environment: Local / Staging / Production
```

```
Total Tests: 27
Passed: ____ (____%)
Failed: ____ (____%)
Skipped: ____

Critical Issues: ____
Medium Issues: ____
Low Issues: ____

Performance:
- Avg Response Time: ____
- Slowest API: _________
- Fastest API: _________

Notes:
_____________________________________________________________________
_____________________________________________________________________
```

Version: 1.0.0
Last Updated: April 30, 2026
Maintained By: Development Team
Status: ☑ Ready for Testing